



nvme ssd firmware fuzzing을 하는데 UART연결해서 live logging하면서 하고있어. 우분투 OS가 freeze 돼 버리고 해당 평가 창에는 Message from syslogd@linux, kernel: [55152.175852] watchdog: BUG: soft lockup - CPU#16 stuck for 28133s! [systemd:1], CPU#12 stuck from samba-bggd:1600 그 외 kthreadd:2, nmbd:1584, AWT-EventQueue 등 다양한게 계속 주기적으로 찍히고 있었어. 그 상황에서의 dmesg는 general protection fault, probably for non-canonical address 0x44507fdc24b37cb0: 0000 [#1] SMP NOPTI, CPU: 4 PID: 322695 Comm: Python3 Tainted: P O 5.15.0-139-generic #149~20.04.1-Ubuntu, Hardware name: ASUS system product name/ROG MAXIMUS Z790 HERO, BIOS 0904 03/29/2023, RIP: 0010:alloc_vmap_area.part.0+0xe8/0x8d0 하고 다양한 주소들이 찍히고 Call Trace 나와 그리고 watchdog: BUG: soft lockup - CPU#6 stck for 26s! [Networkmanager:1267] 이 여기도 찍혀있어. 이후 또 이런 내용이 반복돼. SSD FW의 이슈일지 UART때문인지 모르겠어. 이 원인을 알아낼수있어? 그리고 해결할 방법이 있어? 내가 다 옮겨적지 못한 더 필요한 정보가 있으면 말해

상당히 복잡한 복합 장애 케이스입니다. 로그를 분석해보면 단일 원인이 아닌 연쇄 장애(cascading failure)이며, 근본 원인은 거의 확실하게 특정할 수 있습니다.

근본 원인 분석

핵심 단서: `alloc_vmap_area` + **Python3**

가장 중요한 단서가 이 한 줄입니다:

```
RIP: 0010:alloc_vmap_area.part.0+0xe8/0x8d0
Comm: Python3 Tainted: P O
```

`alloc_vmap_area`는 커널의 **vmalloc** 가상 주소 공간 할당자입니다. Python3 프로세스(퍼징 스크립트)가 이 함수에서 크래시 났다는 것은, **vmalloc** 주소 공간이 고갈(**exhaustion**) 되거나 손상된 상태에서 잘못된 주소(`0x44507fdc24b37cb0` — non-canonical address)에 접근했다는 의미입니다. x86_64에서 canonical address는 `0x0000~0x00007FFF...` 또는 `0xFFFF8000...` 범위인데, 저 주소는 명백히 깨진 포인터입니다.^{[1] [2]}

Tainted: P O 의미:^[3]

- **P**: 독점(Proprietary) 모듈 로드됨 (SSD 벤더 드라이버 또는 퍼징 커널 모듈)
- **O**: Out-of-tree 모듈 (커널 트리 외부 모듈)

즉, 서드파티 커널 모듈이 **vmalloc** 메모리를 오염/고갈시킨 것이 1차 원인입니다.

Soft Lockup은 결과이지 원인이 아님

Soft lockup은 "CPU가 20초 이상 커널 선점 없이 루프를 뒀"을 의미합니다. `systemd`, `NetworkManager`, `kthreadd`, `nmbd`, `AWT-EventQueue` 등 수십 개 프로세스에서 동시에 발생한 것은, 메모리 할당자(allocator)가 완전히 막혀서 모든 프로세스가 메모리 대기 상태로 **stall**된 것입니다. 이것은 2차 증상입니다.^{[4] [5]}

장애 발생 시나리오 (추정 순서)

```
NVMe 퍼저(Python3) 실행
↓ 장시간 fuzzing (무작위 NVMe 커맨드 반복 전송)
↓ 커널 드라이버 내부 vmalloc 반복 할당 → 주소 공간 고갈/단편화
↓ alloc_vmap_area에서 손상된 포인터(non-canonical) 역참조
↓ general protection fault (#GPF) 발생
↓ 커널 메모리 서브시스템 손상
↓ 전체 프로세스 메모리 할당 차단 → 동시다발 soft lockup
↓ OS freeze
```

UART는 별도 하드웨어 시리얼 채널이므로 이 크래시의 직접 원인은 아닙니다. 단, UART 드라이버(`tty/serial`)도 **vmalloc**을 사용하므로 고갈 상태에서 간접 영향을 받을 수 있습니다.^[6]

원인별 확인 방법 (재현 전)

crash 직후 살아있는 시스템이 있다면:

```
# vmalloc 사용량 확인 (단편화 및 고갈 여부)
cat /proc/vmallocinfo | wc -l
cat /proc/meminfo | grep -E "VmallocTotal|VmallocUsed|VmallocChunk"

# 어떤 모듈이 Tainted 상태인지 확인
cat /proc/modules | grep -E "O|P" # out-of-tree, proprietary 모듈

# 직전 커널 oops 전체 확인
sudo journalctl -k --since "1 hour ago" | grep -E "fault|BUG|Oops|vmalloc"

# Python3 프로세스의 메모리 맵 (퍼져 살아있다면)
cat /proc/$(pgrep python3)/maps | grep vmalloc
```

해결 방법

단기 (재현 방지)

1. **vmalloc** 공간 확장 — 커널 부트 파라미터에 vmalloc 크기 명시:^[7]

```
# /etc/default/grub 편집
GRUB_CMDLINE_LINUX="vmalloc=512m" # 기본 128MB → 확장
sudo update-grub && sudo reboot
```

2. 퍼징 주기 제한 — 장시간 연속 실행 대신 세션 분할:

```
# 예: 5분마다 재시작하여 vmalloc 단편화 방지
while true; do timeout 300 python3 fuzz.py; sleep 10; done
```

3. **SPDK userspace NVMe** 드라이버 사용 — 커널 NVMe 드라이버 대신 SPDK로 퍼징하면 kernel vmalloc 압박을 우회할 수 있습니다:^[8]

```
# NVMe 드라이버 언바인드 후 SPDK vfio-pci 바인드
sudo scripts/setup.sh
```

4. **Out-of-tree** 모듈 최소화 — Tainted: P 0 원인인 서드파티 모듈 제거 또는 대체 검토^[3]

중기 (안정적 퍼징 환경)

- 커널 5.15 대신 최신 **LTS(6.8+)** 사용 — vmalloc 관련 버그 다수가 상위 버전에서 패치됨^[9]
- **VM(가상머신)** 격리 환경에서 퍼징 — KVM + VFIO passthrough로 호스트 OS 보호
- **crash dump** 설정 (kdump): 다음 freeze 때 전체 vmcore 수집

```
sudo apt install linux-crashdump
sudo systemctl enable kdump
```

```
# /etc/default/kdump-tools에서 CRASHKERNEL 설정
```

추가로 필요한 로그 정보

현재 로그에서 아직 확인이 안 된 중요 정보들입니다:

항목	확인 명령
전체 Call Trace (GPF 이후)	<code>sudo journalctl -k > dmesg_full.txt</code>
로드된 out-of-tree 모듈명	<code>lsmod \ grep -v "(built-in)"</code> 후 /proc/modules P/O 플래그 확인
Oops 발생 시점의 VmallocUsed	crash 직전 /proc/meminfo 스냅샷
Python3 퍼저의 정확한 구조	SPDK 기반인지, 직접 /dev/nvme0 ioctl인지
UART 드라이버 방식	ttyUSB* vs ttyS*, 어떤 모듈 사용 중인지

Call Trace 전체와 로드된 모듈 목록을 공유해 주시면 더 정확한 원인 모듈을 특정할 수 있습니다.

✱

프리즈돼서 재부팅 할 수 밖에 없었어 현재 확인 가능한 내용들이 아닌거지?

맞아요, 재부팅 후에는 실시간 메모리 상태(/proc/meminfo, /proc/vmallocinfo, 로드된 모듈 등)는 모두 날아갑니다. 하지만 다행히 확인 가능한 것들이 아직 있습니다.

재부팅 후 확인 가능한 것들

재부팅해도 디스크에 기록된 로그는 남아 있습니다:

```
# 1. 커널 로그 전체 (가장 중요 - crash 전후 모두 포함)
sudo journalctl -k -b -1 > ~/crash_dmesg.txt
# -b -1 : 직전 부팅 세션의 로그
cat ~/crash_dmesg.txt | grep -E "fault|BUG|Oops|Call Trace|RIP|alloc_vmap|tainted|loc

# 2. syslog / kern.log (UART로 찍힌 것과 비교 가능)
sudo cat /var/log/syslog.1          # 직전 로그 (rotate된 것)
sudo cat /var/log/kern.log.1

# 3. crash dump이 설정돼 있었다면 (kdump)
ls /var/crash/
```

재부팅 후엔 확인 불가능한 것들

반대로 이건 이미 소멸된 정보입니다:

항목	이유
/proc/meminfo VmallocUsed	프로세스 종료 시 초기화
/proc/vmallocinfo	런타임 메모리 맵, 재부팅 시 리셋
Python3 퍼저 프로세스 메모리 맵	프로세스 소멸
로드된 모듈 상태 (lsmod Tainted)	재부팅 시 클리어

지금 당장 해야 할 것

우선 직전 부팅 로그부터 꺼내세요:

```
sudo journalctl -k -b -1 --no-pager > ~/crash_dmesg_prev.txt
wc -l ~/crash_dmesg_prev.txt    # 로그 라인 수 확인
```

만약 -b -1로 로그가 안 나오면 journald가 persistent logging이 꺼져 있을 수 있습니다. 그 경우:

```
# persistent logging 활성화 (다음 crash를 위해 지금 설정)
sudo mkdir -p /var/log/journal
sudo systemctl restart systemd-journald

# 향후 재부팅 후에도 -b -1 로그가 남음
```

직전 부팅 로그가 남아 있다면 **Call Trace** 전체와 **Tainted** 모듈명을 여기서 확인할 수 있고, 그게 나오면 원인을 더 정확히 특정할 수 있습니다. 결과 공유해 주시면 이어서 분석해 드릴게요.

✻

```
그나마 cat ~/crash_dmesg.txt | grep -E
"fault|BUG|Oops|Call
Trace|RIP|alloc_vmap|tainted|lockup" 에서 나온 정보
가 아까 찍어줬던 정보들인데 Call Trace 부분 추가로 적
어줄게. general protection fault, probably for non-
canonical address 0x44507fdc24b37cb0: 0000 [#1]
SMP NOPTI Call Trace: ?
alloc_vmap_area.part.0+0xe8/0x8d0
alloc_vmap_area+0x51/0x80 ?
exc_page_fault+0x89/0x170 RIP: 0033:0x7fab2075ff3f
RIP: 0010:alloc_vmap_area.part.0+0xe8/0x8d0
watchdog: BUG: soft lockup - CPU#6 stuck for 26s!
[NetworkManager:1267] RIP:
0010:native_queued_spin_lock_slowpath+0x79/0x230
Call Trace: ? softlockup_fn+0x30/0x30
alloc_vmap_area.part.0+0x3ea/0x8d0
alloc_vmap_area+0x51/0x80 RIP:
0033:0x7f8bad9e4345 watchdog: BUG: soft lockup -
CPU#6 stuck for 52s! [NetworkManager:1267]
```

이제 명확하게 원인이 특정됩니다. Call Trace가 정확히 무슨 일이 일어났는지 설명해 줍니다.

정확한 장애 메커니즘

두 개의 Call Trace를 보면 연속된 2단계 장애입니다:

1단계: vmalloc 주소 공간 손상 → GPF

```
RIP: 0010:alloc_vmap_area.part.0+0xe8/0x8d0
general protection fault (non-canonical address 0x44507fdc24b37cb0)
```

퍼저(Python3)가 NVMe 커맨드를 대량으로 던지는 과정에서 Tainted: P 0 서드파티 모듈이 vmalloc 내부 자료구조(RB-tree 노드)를 오염시켰습니다. alloc_vmap_area가 RB-tree 탐색 중 해당 깨진 포인터를 역참조하면서 GPF가 터진 것입니다.^[45]

2단계: vmap_area_lock 글로벌 스핀락 데드락 → 전체 freeze

```
RIP: 0010:native_queued_spin_lock_slowpath+0x79/0x230
Call Trace: alloc_vmap_area.part.0+0x3ea/0x8d0
→ CPU#6 stuck for 26s ... 52s [NetworkManager:1267]
```

1단계 GPF 이후 vmap_area_lock이 해제되지 않은 채로 남겨졌습니다. vmalloc은 전역 스핀락 3개 (vmap_area_lock, free_vmap_area_lock, purge_vmap_area_lock)로 보호되는데, 락을 잡은 채로 크래시가 나면 다른 모든 프로세스가 이 락을 기다리며 무한 대기기에 빠집니다. NetworkManager, systemd, kthreadd, samba, AWT-EventQueue — 이 모든 프로세스가 vmalloc을 요청하는 일반적인 동작 중에 글로벌 스핀락에서 blocked된 것입니다.^{[46] [47] [45]}

장애 흐름 정리

```
[NVMe 퍼저 Python3 + 서드파티 커널 모듈]
  ↓ vmalloc RB-tree 노드 오염
alloc_vmap_area → non-canonical 포인터 역참조
  ↓ GPF (#1) 발생
vmap_area_lock 스핀락 미해제 상태로 중단
  ↓
시스템 전체 vmalloc 요청 → 락 대기 무한 루프
  ↓
모든 CPU에서 동시다발 soft lockup (26s → 52s → ...)
  ↓
OS complete freeze
```

핵심 원인 결론

구분	판정
SSD FW 직접 원인?	가능성 낮음 — SSD FW 버그라면 NVMe 드라이버 레이어에서 터지고 nvme 관련 Call Trace가 찍혀야 함
UART 직접 원인?	아님 — UART는 별도 시리얼 채널, Call Trace에 전혀 등장 안 함
실제 원인	Tainted: P O out-of-tree 커널 모듈이 vmalloc RB-tree를 오염 → vmap_area_lock 데드락 → 시스템 전체 freeze ^[48]

해결 방법

즉시 해야 할 것

```
# 1. 어떤 out-of-tree 모듈이 로드돼 있었는지 확인 (재부팅 후에도 /etc/modules 및 modprobe.d
cat /etc/modules
ls /etc/modprobe.d/
find /lib/modules/$(uname -r)/extra/ -name "*.ko" 2>/dev/null # 외부 설치 모듈
```

```
# 2. 현재 로드된 모듈 중 out-of-tree 확인
cat /proc/modules | awk '$5 == "P" || $5 == "OE" || $5 == "O"'
```

근본 해결책 3가지

① out-of-tree 모듈 제거/교체 (핵심)

Tainted 원인 모듈을 특정해서 제거하세요. 퍼징용 커널 모듈을 직접 작성했다면 `vmalloc/vfree` 짝이 맞는지, 예외 처리 경로에서 락 해제가 누락되지 않았는지 점검해야 합니다.^[48]

② kdump 설정 (다음 재현을 위해 지금 설정)

```
sudo apt install linux-crashdump
# 재부팅 후 grub에 crashkernel=256M 파라미터 자동 추가됨
sudo systemctl enable kdump
# 다음 freeze 시 /var/crash/에 vmcore 덤프 생성 → crash 도구로 정확한 락 보유자 확인 가능
```

③ 퍼징 환경을 VM으로 격리 (장기)

호스트 Ubuntu에서 직접 퍼징하면 커널 모듈 오염이 호스트 전체를 날립니다. KVM + VFIO passthrough로 NVMe를 VM에 넘겨서 퍼징하면 VM이 죽어도 호스트는 살아남습니다.^[49]

지금 바로 확인해야 할 것: 퍼징 스크립트나 관련 커스텀 커널 모듈(.ko 파일)이 있다면 그게 어떤 모듈인지 알려주시면 구체적으로 어느 코드 경로가 문제인지 좁힐 수 있습니다.

✻

modules에는 아무것도 안나오고 그냥 주석부분만 나왔고, modprobe.d에선 `alsa-basse.conf`, `blacklist-ath_pci.conf`, `blacklist-firewire.conf`, `blacklist-modem.conf`, `blacklist-rare-network.conf`, `iwlwifi.conf`, `amd64-microcode-blacklist.conf`, `blacklist.conf`, `blacklist-framebuffer.conf`, `blacklist-oss.conf`, `intel-microcode-blacklist.conf`, `stlink_v1.conf` 이렇게 나왔고 나머지 두 명령에서는 아무것도 안나왔어

중요한 발견입니다. 방향을 수정해야 합니다.

분석 방향 전환: out-of-tree 모듈이 아닐 가능성

/proc/modules에서 out-of-tree(O/P) 모듈이 하나도 없다는 결과가 나왔으므로, Tainted: P 0 플래그는 재부팅 이전 시점에만 존재한 것이고 현재 상태와 다를 수 있습니다. 따라서 원인 후보를 다시 정렬해야 합니다.

현재 상황 재해석

항목	내용
Tainted: P 0	퍼징 실행 당시 로드됐던 커스텀 모듈 (재부팅 후 사라짐)
modprobe.d 목록	표준 blacklist들 + stlink_v1.conf (ST-Link 디버거 관련)
현재 out-of-tree 모듈 없음	퍼징할때만 로드하는 모듈이 있었을 가능성

stlink_v1.conf가 여기서 실마리입니다. ST-Link는 JTAG/SWD 디버거인데, NVMe FW 퍼징 환경에 이게 있다는 건 UART 외에 별도 디버그 인터페이스도 쓰고 있을 가능성이 있습니다.

지금 확인해야 할 것들

1. 퍼징 실행 시 로드하는 모듈이 있는지

```
# 퍼징 스크립트 실행 전/후에 로드되는 모듈 확인
# 다음 번 퍼징 시작 직전에 실행:
lsmod > /tmp/modules_before.txt

# 퍼징 시작 후:
lsmod > /tmp/modules_after.txt
diff /tmp/modules_before.txt /tmp/modules_after.txt
```

2. Python3 퍼저가 어떤 방식으로 NVMe에 접근하는지 확인

```
# 퍼저 실행 중 어떤 syscall/ioctl을 사용하는지
sudo strace -p $(pgrep python3) -e trace=ioctl,mmap,mprotect 2>&1 | head -100

# 또는 퍼저가 어떤 fd를 열고있는지
ls -la /proc/$(pgrep python3)/fd | grep nvme
```

3. vmalloc 사용량 모니터링 (다음 퍼징 세션 시작 시)

```
# 퍼징 중 vmalloc 사용량 지속 모니터링 (freeze 전에 추세 파악)
watch -n 1 'cat /proc/meminfo | grep -E "Vmalloc|Slab|PageTables"'
```

핵심 질문

원인을 정확히 좁히려면 다음 3가지를 알아야 합니다:

1. Python3 퍼저가 직접 작성한 코드인가요, 아니면 **SPDK/libnvme** 등 기존 프레임워크 기반인가요?
2. NVMe에 접근하는 방식이 `/dev/nvme0 ioctl`인지, `/dev/ng0n1 passthrough`인지, 아니면 커스텀 커널 모듈을 통한 것인지요?
3. 퍼징 중 별도로 `insmod/modprobe`로 로드하는 `.ko` 파일이 있나요?

이 세 가지에 따라 원인이 완전히 달라집니다. 특히 2번 — `/dev/nvme0`에 직접 `NVME_IOCTL_ADMIN_CMD`를 대량으로 보내는 경우 커널 NVMe 드라이버 내부에서 `vmalloc` 반복 할당이 발생할 수 있고, 이게 CVE-2026-23244처럼 커널 NVMe 드라이버 자체 버그를 트리거했을 가능성도 있습니다.^[61]

✱

첨부한게 fuzzer 관련 파일이야 보고 vmalloc이든 뭐든 영향 줄 수 있는 포인트가 뭔지 봐봐

이제 퍼저 코드를 직접 분석했으니 정확한 문제 포인트를 짚을 수 있습니다. 생각보다 훨씬 구체적인 원인이 여러 개 보입니다.

코드 분석 결과: **vmalloc** 장애 원인 포인트

❏ 원인 1 (가장 유력): `nvmekerneltimeoutsec` 극단적 설정

config를 보면:^[76]

```
"nvmekerneltimeoutsec": 2592000    // = 30일(!)
"nvmeassthrotimeouts": 2592000000  // = 30일
```

퍼저가 시작할 때 이 코드를 실행합니다:^[77]

```
# /sys/module/nvme_core/parameters/admin_timeout
# /sys/module/nvme_core/parameters/io_timeout
p.write_text(str(value)) # 2592000초를 커널 파라미터에 씀
```

`nvme_core` 커널 모듈의 타임아웃을 30일로 설정하면, NVMe 커맨드가 hang될 때 커널이 타임아웃 처리를 못 하고 해당 요청을 잡고 있는 내부 구조체(DMA 버퍼, SQ/CQ 매핑)가 `vmalloc` 공간을 점유한 채로 누적됩니다. 퍼징 중 수천 개의 커맨드가 중간에 hang되면 이것이 `vmalloc`을 천천히 고갈시킵니다.^[76]

❖ 원인 2: subprocess.Popen 반복 호출 → 프로세스 메모리 누수

`send_nvme_command()`마다 `subprocess.Popen(["nvme", ...])`를 새로 fork합니다. `nvme-cli`는 실행 시 `/dev/nvme0`에 대해 내부적으로 `ioctl(NVME_IOCTL_ADMIN_CMD)`를 호출하는데, 이 `ioctl`은 커널이 DMA-able 메모리를 `vmalloc`으로 할당합니다. 정상 종료 시 해제되지만:^[77]

- 타임아웃 케이스: `process.communicate(timeout=...)` 후 `crash_nvme_pid()`로 kill하면, 커널 입장에서 진행 중인 NVMe I/O의 DMA 버퍼가 아직 in-flight 상태일 수 있습니다
- 비정상 종료 경로: `except OSError` 등에서 `process.kill()` 후 커널 NVMe 드라이버가 cleanup을 못 하면 해당 `vmalloc` 영역이 leak됩니다

❖ 원인 3: OpenOCD + J-Link 동시 실행 중 PCIe 전원 조작

`power_cycle()/POR` 로직에서 PCIe 링크를 내렸다 올리는데:^[77]

```
# PCIe rescan + NVMe re-attach
subprocess.run(["echo", "1", ">", f"/sys/bus/pci/devices/{bdf}/remove"])
subprocess.run(["echo", "1", ">", "/sys/bus/pci/rescan"])
```

PCIe 핫플러그 중 `nvme_core` 드라이버가 기존 `vmalloc` 영역을 free하고 재할당하는 과정에서 `vmap_area_lock`을 잡은 채로 PCIe 인터럽트가 끼어들면 RB-tree가 오염될 수 있습니다.

❖ 원인 4: Tainted: P 0 실제 출처 — UFAS 또는 J-Link 드라이버

`config`에 `ufas` 바이너리와 `JLinkExe`가 있습니다. J-Link는 실행 시 `/dev/jlink` 또는 `libjaylink`를 통해 out-of-tree USB 커널 드라이버를 로드할 수 있습니다. 이것이 Tainted: P 0의 실제 원인일 가능성이 높습니다. `stlink_v1.conf`가 `modprobe.d`에 있는 것도 이와 연관됩니다.^{[78] [76]}

즉시 적용 가능한 수정

Fix 1: 커널 타임아웃을 현실적인 값으로

```
// fuzzer_config-1.json
"nvmekerneltimeoutsec": 30
```

30일 → 30초로 변경. 퍼징 중 hang된 커맨드를 커널이 정상적으로 abort하고 `vmalloc` 자원을 회수하게 됩니다.^[76]

Fix 2: nvme-cli 프로세스 타임아웃 후 NVMe 리셋 추가

현재는 타임아웃 시 `process.kill()`만 합니다. 커널 NVMe 드라이버에게도 명시적으로 리셋을 알려야 합니다:

```
# 타임아웃 후 nvme reset 호출
sudo nvme reset /dev/nvme0
```

```
# 또는 PCIe FLR
echo 1 > /sys/bus/pci/devices/<BDF>/reset
```

Fix 3: vmalloc 모니터링 스레드를 퍼저에 추가

```
import threading, pathlib

def vmalloc_watchdog():
    while True:
        info = pathlib.Path("/proc/meminfo").read_text()
        for line in info.splitlines():
            if "VmallocChunk" in line:
                chunk_kb = int(line.split()[^5_1])
                if chunk_kb < 65536: # 64MB 미만이면 경고
                    log.warning(f"VMALLOC LOW: largest free chunk = {chunk_kb} kB")
                time.sleep(10)

threading.Thread(target=vmalloc_watchdog, daemon=True).start()
```

Fix 4: kdump 설정 (다음 크래시 분석용)

```
sudo apt install linux-crashdump
sudo systemctl enable kdump
# 다음 freeze 시 /var/crash/에 vmcore 저장됨
sudo journalctl -k -b -1 # 이후 재부팅에서 이전 로그 확인 가능
```

요약

원인	코드 위치	위험도
nvmekerneltimeoutsec=2592000 (30일) → vmalloc 누적	fuzzer_config-1.json + configure_nvme_timeouts()	☒ 최우선
Popen 반복 fork + 비정상 종료 시 DMA 버퍼 미해제	send_nvme_command()	☒ 높음
PCIe POR 중 vmap_area_lock 경쟁	power_cycle() / POR 로직	☒ 중간
J-Link/UFAS out-of-tree 드라이버 → Tainted P O	JLinkExe USB 드라이버	☒ 중간

가장 먼저 nvmekerneltimeoutsec를 30으로 낮추고 다시 실행해보세요. 이것만으로도 vmalloc 고갈 속도가 극적으로 줄어든 것입니다. ^{[76] [77]}

✱

1. https://bugzilla.kernel.org/show_bug.cgi?id=205601
2. <https://us-west-2b-production-gl-awsz-arm.com/linux-arm/ch-kernel-dpu/-/commit/afd07389d3f4933c7f7817a92fb5e053d59a3182>
3. <https://access.redhat.com/solutions/7056610>
4. <https://access.redhat.com/solutions/7035748>

5. <https://docs.kernel.org/admin-guide/lockup-watchdogs.html>
6. <https://docs.byteplus.com/ko/docs/ecs/fio-command>
7. <https://lists.infradead.org/pipermail/linux-riscv/2024-June/055602.html>
8. <https://spdk.io/news/2023/04/21/fuzzing/>
9. <https://github.com/spdk/spdk/issues/2277>
10. <https://velog.io/@provence/우분투-watchdog-BUG-soft-lockup-해결하기>
11. <https://www.sangchul.kr/1037>
12. <https://github.com/spdk/spdk/issues/1222>
13. <https://jaehyojjang.dev/트러블슈팅/2023-06-10-linux-cpu-error/>
14. <https://www.mail-archive.com/debian-bugs-closed@lists.debian.org/msg822307.html>
15. <https://www.scribd.com/document/644748649/UART-해킹-Advanced>
16. <https://bbs.archlinux.org/viewtopic.php?id=271413>
17. <https://patchew.org/linux/20260309062840.2937858-2-iam@sung-woo.kim/>
18. <https://www.digibeatix.com/ubuntu/ko/troubleshooting-ko/fix-network-issues-ubuntu/>
19. https://bugzilla.kernel.org/show_bug.cgi?id=206075
20. <https://access.redhat.com/solutions/7107913>
21. https://www.reddit.com/r/btrfs/comments/q12zbl/system_keeps_freezing_after_wakeup_with_a_new_ssd/
22. <https://git.zx2c4.com/linux-rng/commit/mm?id=55ccad6fc1a03c814c26f0e6b35db50feda2c59e>
23. <https://git.zx2c4.com/linux-rng/commit/mm/vmalloc.c?id=68ac546f265ba36cd4f29c77b3841fb777315581>
24. <https://telematik.prakinf.tu-ilmenau.de/gitlab/pub/spdk/-/commit/93351b0e01f5732fc61346d49f4927d9d286dad0>
25. <https://stackoverflow.com/questions/40936396/rapid-insmod-and-rmmod-leads-to-vmalloc-allocation-failure-is-this-a-memory-leak>
26. <https://github.com/benlimpa/nvme-fuzzing-report>
27. https://www.usenix.org/system/files/sec22summer_scharnowski.pdf
28. <https://sre.ring0.de/linux/tree/include/linux/vmalloc.h?h=v5.11-rc7&id=e24f9c5f6e3127a0679d5ba5575a181b80f219c9>
29. https://github.com/spdk/spdk/blob/master/test/app/fuzz/nvme_fuzz/README.md
30. <https://docs.kernel.org/admin-guide/lockup-watchdogs.html>
31. <https://www.sangchul.kr/1037>
32. <https://access.redhat.com/solutions/6807441>
33. <https://lists.infradead.org/pipermail/linux-riscv/2024-June/055461.html>
34. <https://gem5.googlesource.com/arm/linux/+0eb96bf754d7fa6635aa0b0f6650c74b8a6b1cc9/kernel/watchdog.c>
35. <https://www.mail-archive.com/kernel-packages@lists.launchpad.net/msg494568.html>
36. <https://lwn.net/Articles/7473/>
37. <https://access.redhat.com/solutions/7035748>
38. <https://forums.developer.nvidia.com/t/kernel-hang-on-large-nvme-writes-ext4-soft-lockup-in-ext4-buffered-write-iter-bug-at-inode-c-2176-asus-ascent-gx10-kernel-6-17-0-1014-nvidia/367605>
39. <http://lists.infradead.org/pipermail/linux-riscv/2024-June/055596.html>

40. <https://docs.byteplus.com/ko/docs/ecs/fio-command>
41. <https://arxiv.org/abs/2505.03062>
42. <https://github.com/benlimpa/nvme-fuzzing-report>
43. <https://www.themoonlight.io/en/review/testing-ssd-firmware-with-state-data-aware-fuzzing-accelerating-coverage-in-nondeterministic-io-environments>
44. <https://github.com/spdk/spdk/issues/1222>
45. <https://lwn.net/Articles/932396/>
46. <https://access.redhat.com/solutions/6978268>
47. <https://access.redhat.com/solutions/658783>
48. <https://vuldb.com/?id.338260>
49. <https://spdk.io/news/2023/04/21/fuzzing/>
50. <https://kernel-internals.org/mm/vmalloc/>
51. <https://stackoverflow.com/questions/75455207/kernel-code-native-queued-spin-lock-slowpath-function>
52. <https://junsoolee.gitbook.io/linux-insides-ko/summary/syncprim/linux-sync-2>
53. <https://retrace.fedoraproject.org/faf/reports/427333/>
54. <https://lwn.net/Articles/956590/>
55. <https://access.redhat.com/solutions/7077348>
56. https://android.googlesource.com/kernel/common/+refs/tags/android13-5.15-2024-06_r8/mm/vmalloc.c
57. <https://lists.infradead.org/pipermail/linux-riscv/2024-June/055602.html>
58. <https://patchew.org/linux/20240222120536.216166-1-rulin.huang@intel.com/>
59. https://www.reddit.com/r/cpp_questions/comments/156uuz6/what_is_native_queued_spin_lock_slowpathpart0/
60. https://static.sched.com/hosted_files/ossna2025/85/Enhancing_Scalability_of_the_Vmalloc_Mechanism_in_the_Linux_Kernel.pdf
61. <https://www.sentinelone.com/vulnerability-database/cve-2026-23244/>
62. https://www.youtube.com/watch?v=Aa8xIfikh_E
63. <https://discuss.linuxcontainers.org/t/nmi-watchdog-bug-soft-lockup-cpu-stuck/321>
64. <https://lists.opensuse.org/archives/list/kernel-bugs@lists.opensuse.org/thread/IBG6WCUS7DF3R2Z5LSHFOLWZZO5DETOO/>
65. <https://github.com/moby/moby/issues/36675>
66. <https://github.com/falcosecurity/falco/issues/3637>
67. <https://forum.proxmox.com/threads/nmi-watchdog-bug-soft-lockup-cpu-5-stuck-for.32862/>
68. <https://github.com/spdk/spdk/issues/1222>
69. <https://knowledge.broadcom.com/external/article/170185/error-kernel-bug-soft-lockup-cpu-stuck.html>
70. https://blog.csdn.net/Baron_ND/article/details/107959961
71. <https://knowledge.broadcom.com/external/article/397009/error-watchdog-bug-soft-lockup-cpu-stuck.html>
72. <https://access.redhat.com/solutions/5042601>
73. <https://docs.kernel.org/admin-guide/lockup-watchdogs.html>
74. <https://github.com/koverstreet/bcachefs/issues/388>

75. <https://access.redhat.com/solutions/7035748>

76. fuzzer_config-1.json

77. pc_sampling_fuzzer_v8.5.py

78. nvme_seeds-1.py

79. pc_sampling_fuzzer_v8.4.md